

Demo Day

How to show all three assignments

14 July 2026

Contents

1 Demo day — how to show the three assignments	1
1.1 The one-page map	1
1.2 Before you walk in	1
1.3 Assignment 1 — Dhaka Pathfinder	2
1.4 Assignment 2 — Fuel-crisis CSP	2
1.5 Assignment 3 — the two parts	3
1.6 Two things to mention about the template	4
1.7 If something breaks in the room	5

1 Demo day — how to show the three assignments

Everything below works **offline**. The OpenStreetMap graph and the station data are already cached as .pkl files in 1/data/ and 2/data/, so nothing downloads while he is watching. The plots are already generated too. Nothing here depends on the network.

Do a **full dry run the night before**, on the machine and the screen you will actually use. The single most common way this goes wrong is a laptop that has not run the code since March.

1.1 The one-page map

	Assignment	Where	How to show it
1	Search — Dhaka Pathfinder (BFS/DFS/UCS/Greedy/A*/Weighted A*)	1/	Streamlit map UI
2	Constraint satisfaction — fuel-crisis allocator	2/	Streamlit UI
3	Swarm + decision making	3/	Terminal + plots
—	The written submission covering all three	3/report/main.pdf	Hand him this

1.2 Before you walk in

```
# once, to be sure everything still runs
cd 1 && ./run.sh test && cd ..
cd 2 && ./run.sh test && cd ..
cd 3 && ./run.sh test && cd ..
```

```
# build the report
cd 3 && ./run.sh report          # -> 3/report/main.pdf
```

Then **fill in your name and email** in 3/report/main.tex (lines 8 and 9). They currently say Your Full Name and your.email@example.com. Rebuild after you edit.

1.3 Assignment 1 — Dhaka Pathfinder

```
cd 1
./run.sh ui          # opens a Streamlit map in the browser
```

Pick a source and destination on the map, change the traveller context (gender, time of day, weather, vehicle), and show that **the route changes**. That is the whole point of the assignment: the cost model is not distance, it is what it actually costs *that person* to make *that trip*.

The number to have ready if he asks how the algorithms compare:

“A*, UCS and Weighted A* all return the same optimal cost, 212,375 — that is the check that our heuristic is really admissible. Greedy Best-First expands **169** nodes where A* expands **13,362**, so it is about eighty times cheaper, and it pays for it with a route that is 38% worse. DFS is a disaster on a graph with 28,000 nodes: its median path costs eleven times optimal and it fails outright on 42 of 350 runs.”

If the UI is slow to start, fall back to the pre-generated figures in 1/results/plots/.

The weakness to own before he finds it: every edge attribute (traffic, safety, flooding) is *synthetic* — generated deterministically from OSM tags, not measured. Say that yourself. The algorithms are real; the data is plausible, not surveyed.

1.4 Assignment 2 — Fuel-crisis CSP

```
cd 2
./run.sh ui          # Streamlit UI, the easiest thing to show
./run.sh experiments  # if he wants to see the sweep run live
```

The story is the combinatorial blow-up:

“Basic backtracking expands 20 nodes at N=10 and **220,763** at N=50 — four orders of magnitude for a five-fold increase in size, and at N=50 it never terminates on its own, it just hits our time cap. Add forward checking with MRV and it expands **15,880**, a 93% reduction, and it is the only complete solver still finding full assignments at that size.”

Two things you should raise *yourself*, because they look like mistakes if he finds them first:

- **Min-conflicts does zero backtracks at every problem size.** That is not a bug and it is not an achievement — it is definitional. Min-conflicts never builds a partial assignment, so it has nothing to unwind. It starts complete and broken, and repairs.
- **Plain MRV scores a worse objective than basic backtracking at N=50** (4,439 vs 3,542) despite expanding four times fewer nodes. Reason: it leaves 40% of vehicles unassigned against 29%, and the unassigned penalty (w=100) dominates the objective. So our J confounds “quality of the assignment” with “how much of the problem was attempted.” That is a flaw in our metric design, and we say so in the report.

Note: 2/results/REPORT.md is **stale** and its tables disagree with the shipped CSVs. Use the CSVs (2/results/experiments_summary.csv) and the report PDF. Do not open REPORT.md in front of him.

1.5 Assignment 3 — the two parts

This is the assignment with two parts, so it needs the most planning. Budget **ten minutes**: one to frame it, four for Part A, five for Part B.

1.5.1 Set up before he sits down

```
cd 3
./run.sh rl > /tmp/partB.txt      # 4 minutes. Do this EARLY. You will read from the file.
```

Have these open and ready to switch between, in this order:

1. A terminal, sitting in 3/, cleared.
2. results/plots/spatial_swarm.png
3. results/plots/collective_behaviour.png
4. /tmp/partB.txt (or just results/rl_full_run.log, which is the same output)
5. results/plots/rl_policy_maps.png
6. results/plots/rl_model_mismatch.png
7. report/main.pdf, as the fallback for any number you are asked for.

1.5.2 Open with the framing (30 seconds)

Say this first, because it makes the two parts look like one assignment instead of two homeworks stapled together:

"The assignment had two parts, so I built two problems, both set in the same University of Dhaka residential hall. Part A places the hall's Wi-Fi with a swarm. Part B runs the hall's water pump with an MDP. Same building, two different kinds of hard."

1.5.3 PART A — the swarm (about 4 minutes)

Run it live. It only takes 30 seconds, and running something live is worth a lot.

```
./run.sh swarm
```

Then show spatial_swarm.png and explain the problem in one breath:

"40 rooms, concrete walls, and I can afford 3 access points. The red stars are where the swarm put them. The orange lines are the swarm's best-known solution moving over time. The reason this needs a swarm and not calculus is the walls: every time the signal crosses one it drops 8 dB in a step, so the objective is piecewise-constant. There is no gradient to descend."

Then show collective_behaviour.png. This is the figure the whole part exists for. Do not rush it.

"Every point on this chart costs exactly the same amount of computation — 3,030 evaluations. One particle on its own scores 76.8. Thirty particles that I have *forbidden from communicating* — I set the social coefficient c_2 to zero — score 87.3. Random search scores 87.4. They are the same.

Thirty particles that share a single number, the global best, score 89.9.

So the population is not what helps. The communication is. That is the ant-in-a-bowl point from your briefing, and this is the experiment that proves it."

If he pushes on fairness, you have the answer ready: the budget is identical in every bar, and the code *asserts* it at runtime — `assert problem.n_fitness_calls == cfg.n_evals`.

Have in your pocket, if asked: PSO beats random by 2.47 points, Wilcoxon $p = 3.05e-05$ over 15 seeds; and PSO connects all 40 rooms where random search strands one.

1.5.4 PART B — the decision making (about 5 minutes)

Show the printed table from `/tmp/partB.txt` (the E2 block). Frame the problem:

"Same hall, water tank on the roof, electric pump. Load-shedding kills the power worst in the evening, exactly when everyone wants a shower. There is a diesel generator, but the hall buys only two hours of diesel a day. So you have to pump *before* the power goes, and you have to not waste the diesel."

Then prove the problem is worth solving at all, which is the first thing he will wonder:

"The formally-correct greedy policy — that is value iteration with gamma set to zero, not a strawman — loses by 121%. Even the best tuned two-threshold rule, with the generator and both thresholds grid-searched, still loses 14.5%. So the lookahead is doing real work."

Show `rl_policy_maps.png`. Point at the top-right panel:

"Hour of day across, tank level up. Red is 'burn diesel'. Watch the red region *grow* as you enter the shaded evening window — at 7am with a low tank it idles, at 7pm with the same tank it burns diesel, because it knows the outage will be long. That is the agent anticipating."

Bottom row is Q-learning. Same shape, but speckled. That is what 'has not seen enough data yet' looks like."

If he wants it sharper, the witness state is in the printout (E6):

"At 2pm with a 4/10 tank, pumping costs 0.97 *more* that hour than idling. The optimal policy pumps anyway, because the action is worth +4.69 overall. Over five of those points come purely from the future."

Show `rl_model_mismatch.png`. This is the one he will remember, and it is a negative result about our own idea. Lead with that, do not bury it.

"Here is where the assignment actually asks 'why this algorithm in this setting'. I gave Value Iteration a *deliberately wrong* model — I told it outages are rarer than they really are, which is exactly what Dhaka's published load-shedding schedule does — and then scored its policy in the real hall."

I expected Q-learning to win. It does not. A planner whose model is wrong by a factor of four still beats Q-learning after 720,000 steps. Then I added the baseline that could have destroyed my whole thesis: I estimated a model from Q-learning's *own samples* and planned on that. It beat Q-learning by a factor of ten on identical data.

So the honest answer to 'which one, and when' is: if you can write the transition structure down at all, write it down and plan — a roughly-right model is worth more than 82 simulated years of experience. Model-free is what you reach for when you *cannot write the model down*, not when the model is merely wrong."

1.5.5 The sentence to close on

"The two parts answer two different questions. Part A shows that the *communication* is what makes a population work. Part B shows that *having a model at all* beats not having one, even a bad model — which is not what I set out to prove."

That is a student who ran the experiment that could have embarrassed him, and reported it. It is the best thing you have.

1.6 Two things to mention about the template

He said he would provide the report template, and he did. Two things worth telling him:

1. **It does not compile as shipped.** Line 19 of `main.tex` has a bare `\hline`, which is a LaTeX error outside a tabular (Misplaced `\noalign`) and stops the build dead. We replaced it with `\noindent\rule{\textwidth}{0.4pt}`.
2. **references.bib has duplicate keys** — `russell2021ai` and `bishop2006prml` are each defined twice, which makes BibTeX emit "Repeated entry".

Neither is a big deal, but every student in the class will hit both, and pointing them out is the kind of thing that reads well.

1.7 If something breaks in the room

- Streamlit will not start → show the pre-generated PNGs in `1/results/plots/` and `2/results/plots/`. They tell the same story.
- Anything in lab 3 is slow → all its figures are already in `3/results/plots/`. Nothing needs to run live.
- He asks for a number you do not have → it is in `3/report/main.pdf`. Open it and find it, out loud. Looking something up is not a failure; inventing a number is.